

STAT 8020 R Session 15: Spatial Interpolation II

Whitney Huang, Clemson University

Contents

Rainfall Data from Parana State, Brazil	1
Loading and Summarizing the Data	1
Exploring Large-Scale Spatial Trend	3
Variogram Analysis	5
Parameter Estimation	9
MLE	11
Spatial Prediction	12
Conditional Simulations	15

Rainfall Data from Parana State, Brazil

In this R session, we analyze dry-season rainfall data from Paraná State, Brazil. The dataset contains total rainfall measurements from 143 weather stations during May and June, which correspond to the dry season in this region.

The goal of this section is to introduce the spatial structure of the data and motivate the use of Gaussian process spatial interpolation. In particular, we first visualize the observed rainfall values and then examine whether there is evidence of a large-scale spatial trend.

Loading and Summarizing the Data

```
# Load the geoR package, which contains the Paraná rainfall dataset  
# and functions for geostatistical analysis.  
library(geoR)  
  
# Load the Paraná rainfall dataset.  
data(parana)  
  
# Display a summary of the dataset.  
# This gives basic information about the coordinates, rainfall values,  
# and other components stored in the geoR object.  
summary(parana)
```

```
## Number of data points: 143  
##
```

```
## Coordinates summary
##      east    north
## min 150.1220  70.3600
## max 768.5087 461.9681
##
## Distance summary
##      min      max
##  1.0000 619.4925
##
## Borders summary
##      east    north
## min 137.9873  46.7695
## max 798.6256 507.9295
##
## Data summary
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 162.7700 234.1900 269.9200 274.4106 318.2300 413.7000
##
## Other elements in the geodata object
## [1] "loci.paper"
```

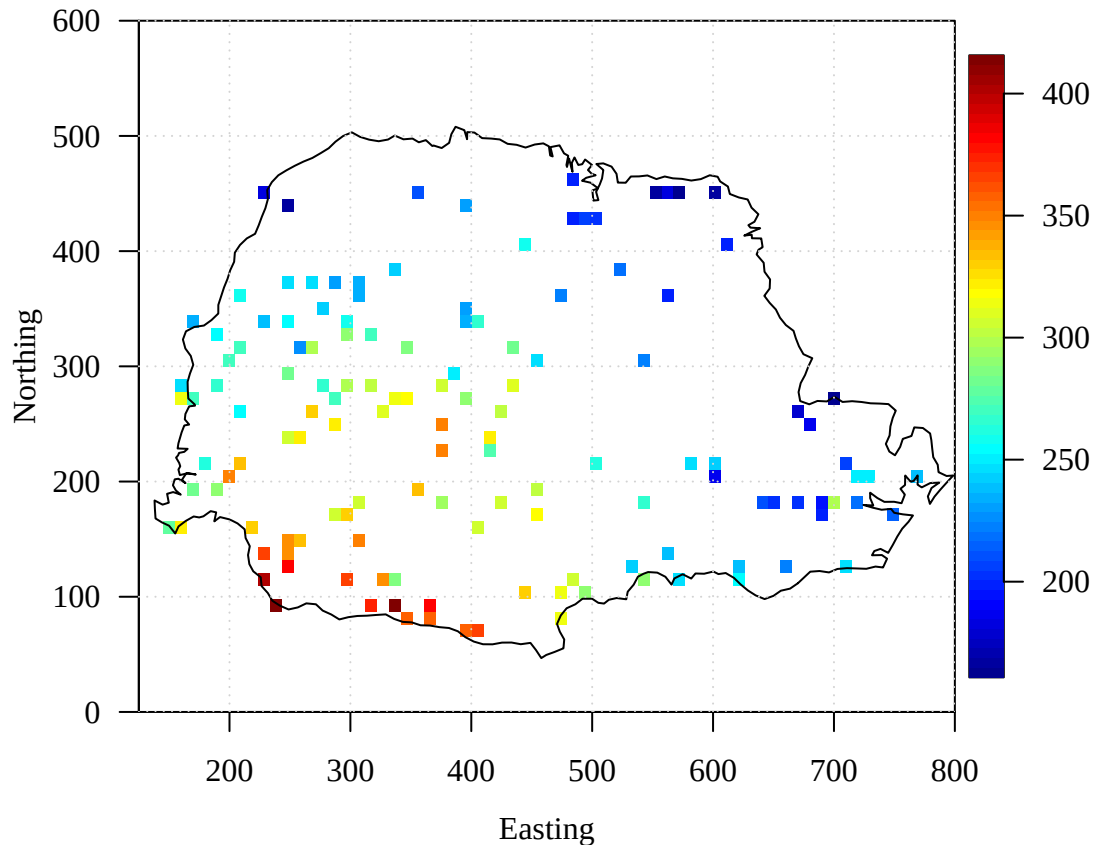
```
# Load the fields package for spatial visualization.
library(fields)

par(las = 1, mar = c(3.5, 3.65, 1, 0.5), mgp = c(2.5, 1, 0), family = "serif")

# Create a spatial plot of rainfall observations.
# quilt.plot produces a color-coded spatial display for irregularly
# spaced spatial data.
# parana$coords contains the station coordinates.
# parana$data contains the observed rainfall values.
quilt.plot(parana$coords, parana$data,
  ny = 36, ylim = c(0, 600), xlim = c(125, 800),
  xlab = "Easting", ylab = "Northing")

# Add the boundary of Paraná State to the plot.
lines(parana$borders)

# Add grid lines to make the spatial location easier to read.
grid()
```



The resulting map shows the spatial locations of the weather stations and the observed rainfall amounts. The color scale allows us to see whether nearby stations tend to have similar rainfall values. This first plot is useful for identifying possible spatial patterns, unusual observations, and areas with sparse station coverage.

Exploring Large-Scale Spatial Trend

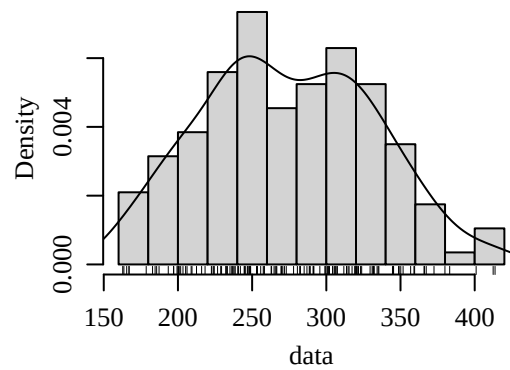
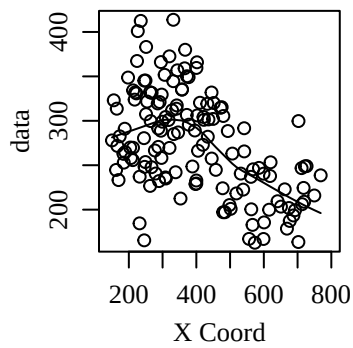
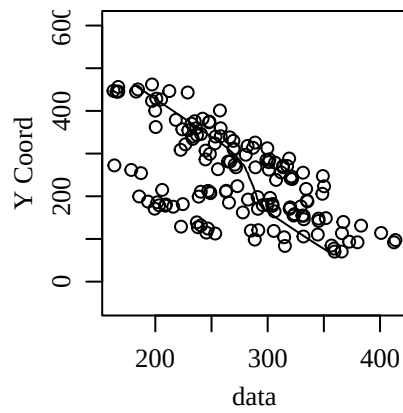
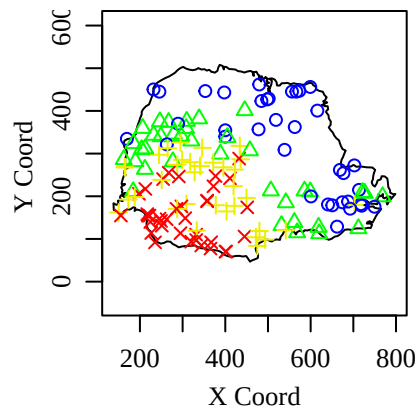
Before fitting a spatial covariance model, it is important to examine whether the rainfall process has a large-scale trend. A spatial trend means that the expected rainfall changes systematically with location.

For example, rainfall may increase or decrease from west to east, or from south to north. If such a trend exists, it should be included in the mean function of the Gaussian process model rather than treated as purely random spatial variation.

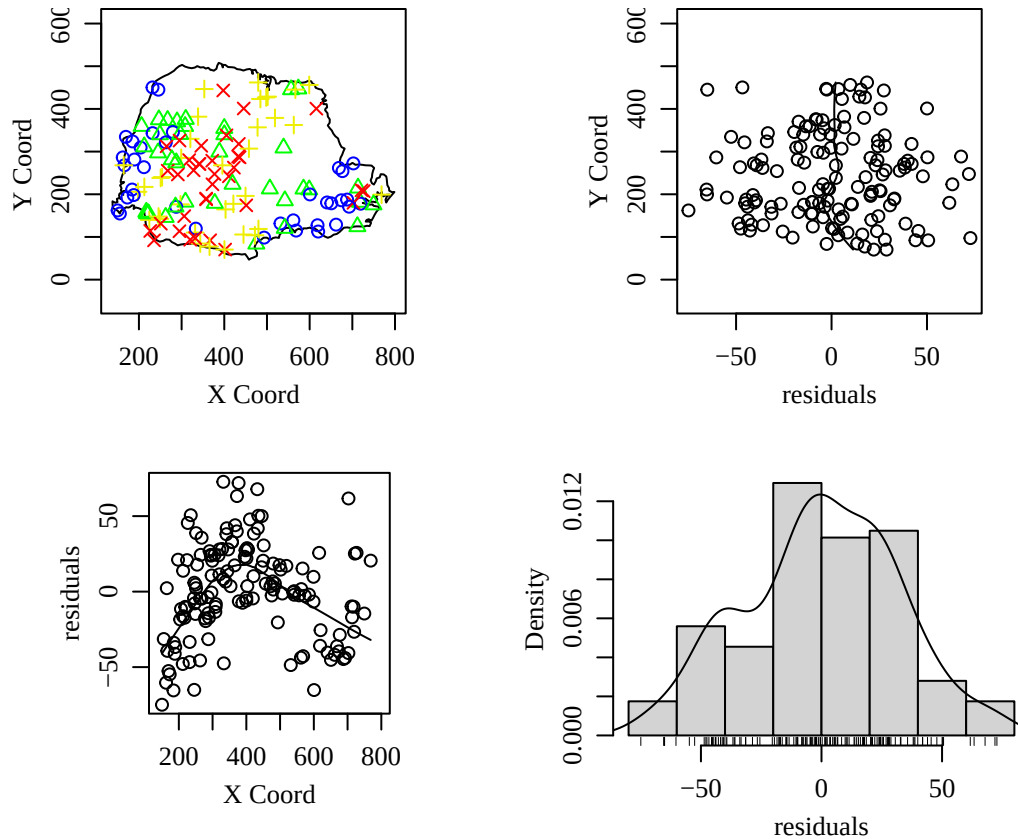
In the following plots, we examine rainfall against the spatial coordinates. The LOWESS curves provide a nonparametric smooth estimate of the relationship between rainfall and each coordinate. These curves help us assess whether a simple linear trend in space is reasonable.

```
par(mar = c(3.5, 4.25, 1.5, 0.5), mgp = c(2.5, 1, 0), family = "serif")

# Plot the geoR object with LOWESS smoothers.
# This gives exploratory plots of rainfall values against spatial coordinates.
# The LOWESS curves help reveal possible nonlinear or linear spatial trends.
plot(parana, lowess = TRUE)
```



```
# Repeat the exploratory plot after specifying a first-order spatial trend.
# trend = "1st" corresponds to a linear trend in the spatial coordinates.
# This helps assess whether a linear mean model is adequate for the data.
plot(parana, trend = "1st", lowess = TRUE)
```



The exploratory plots suggest that rainfall changes systematically with both spatial coordinates. The LOWESS smoothers appear approximately linear, which supports using a first-order spatial trend in the mean model.

Therefore, a reasonable Gaussian process model for these data is

$$Y(s) = \beta_0 + \beta_1 x_1(s) + \beta_2 x_2(s) + \epsilon(s),$$

where $x_1(s)$ and $x_2(s)$ are the spatial coordinates of location s , and $\epsilon(s)$ is a spatially correlated residual process.

In this model, the linear trend captures large-scale spatial variation, while the covariance function for $\epsilon(s)$ captures local spatial dependence.

Variogram Analysis

After exploring the raw rainfall values and the possible large-scale trend, we now examine the spatial dependence structure using empirical variograms. A variogram measures how different observations become as the distance between locations increases. If nearby stations tend to have similar rainfall values, the semivariogram should be small at short distances and increase as distance increases.

We first compare the variogram cloud with the binned empirical variogram. The variogram cloud shows all pairwise squared differences, while the binned variogram summarizes these pairwise differences by distance classes.

```
# Compare the variogram cloud with the binned empirical variogram.
par(mfrow = c(1, 2), las = 1, mgp = c(3, 1, 0), family = "serif")

parana.vario.cloud <- variog(parana, max.dist = 300, cex = 0.5, option = "cloud")
```

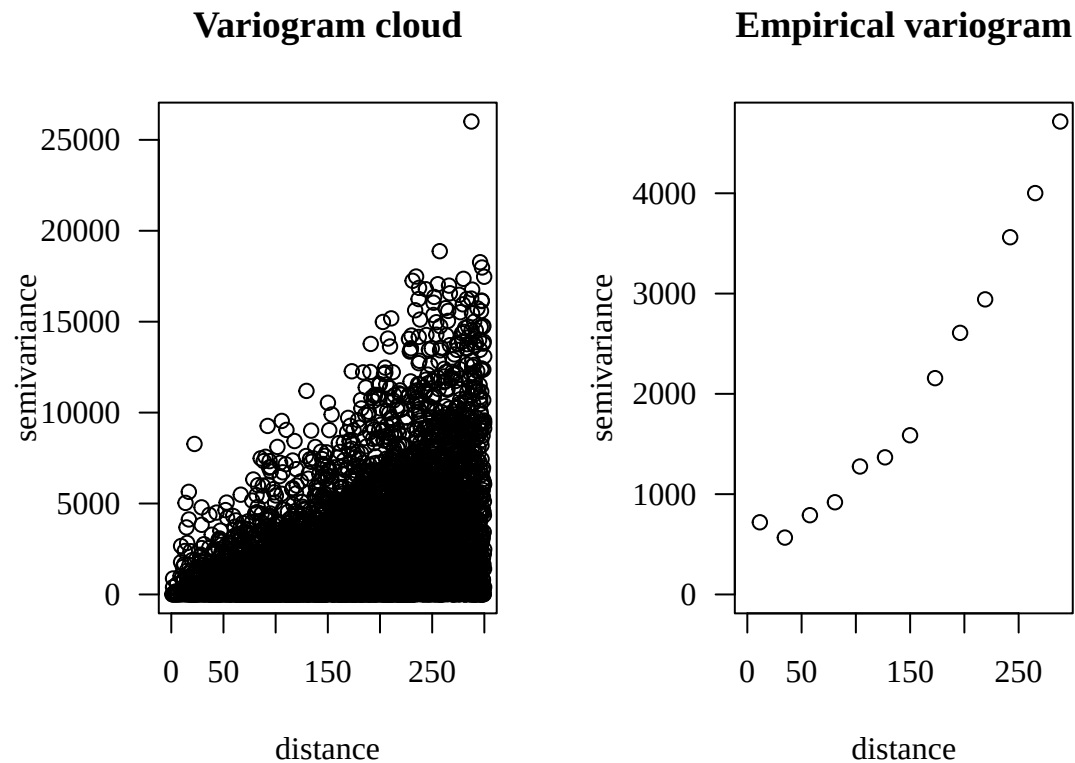
```
## variog: computing omnidirectional variogram
```

```
plot(parana.vario.cloud, main = "Variogram cloud")
```

```
parana.vario <- variog(parana, max.dist = 300, cex = 0.5)
```

```
## variog: computing omnidirectional variogram
```

```
plot(parana.vario, main = "Empirical variogram")
```



The variogram cloud displays substantial variability because it includes all pairs of observations. The binned empirical variogram is easier to interpret because it averages pairwise differences within distance intervals. An increasing variogram pattern suggests that rainfall values are more similar at nearby locations and less similar at distant locations.

Next, we compare the empirical variogram before and after removing a first-order spatial trend. This is important because large-scale trend and local spatial dependence can be confounded. If a spatial trend is present but not removed, the variogram may reflect both the trend and the residual spatial dependence.

```
# Compare empirical variograms before and after removing a first-order spatial trend.
```

```
par(mfrow = c(1, 2), las = 1, mgp = c(3, 1, 0), family = "serif")
```

```
parana.vario <- variog(parana, max.dist = 300, cex = 0.5)
```

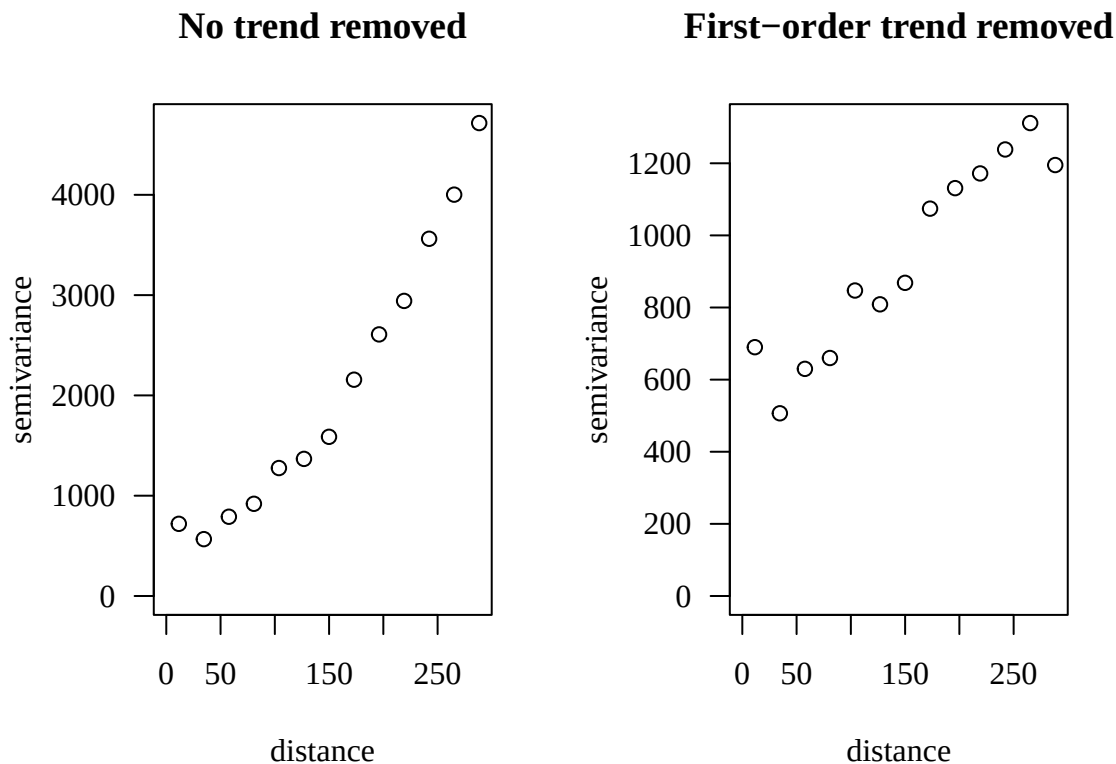
```
## variog: computing omnidirectional variogram
```

```
plot(parana.vario, main = "No trend removed")

parana.variot <- variog(parana, trend = "1st", max.dist = 300, cex = 0.5)
```

```
## variog: computing omnidirectional variogram
```

```
plot(parana.variot, main = "First-order trend removed")
```



The left panel shows the variogram computed from the original rainfall observations. The right panel shows the variogram computed after removing a first-order spatial trend. If the trend-adjusted variogram is lower or has a different shape, this indicates that part of the spatial variation was due to large-scale spatial trend rather than residual spatial correlation.

To formally assess whether the observed variogram indicates meaningful spatial dependence, we compare it with a Monte Carlo envelope generated under the assumption of spatial independence. The envelope represents the range of variograms we might expect if the rainfall values were randomly assigned to the observed locations.

```
# Monte Carlo envelope for testing spatial independence.
par(las = 1, family = "serif")

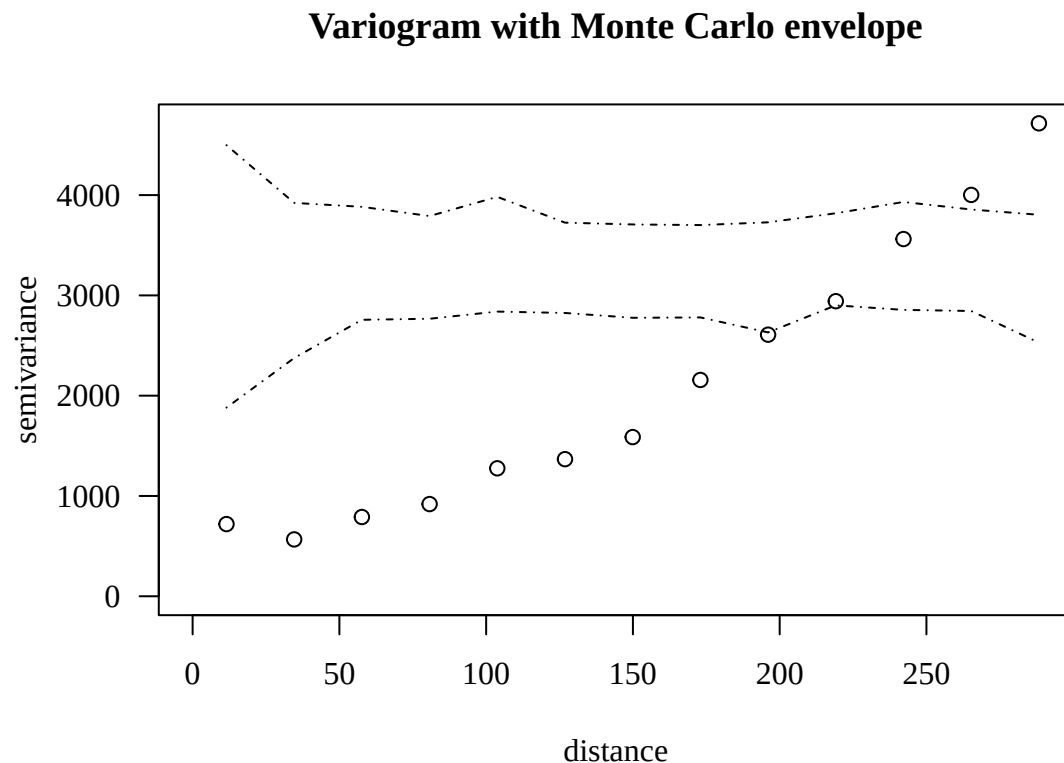
parana.v <- variog(parana, max.dist = 300)
```

```
## variog: computing omnidirectional variogram
```

```
parana.v.env <- variog.mc.env(parana, obj.variog = parana.v)
```

```
## variog.env: generating 99 simulations by permutating data values
## variog.env: computing the empirical variogram for the 99 simulations
## variog.env: computing the envelopes
```

```
plot(parana.v, env = parana.v.env, main = "Variogram with Monte Carlo envelope")
```



If the empirical variogram falls outside the Monte Carlo envelope, especially at short distances, this provides evidence against spatial independence. In this context, it suggests that rainfall values are spatially correlated and that a spatial covariance model is appropriate.

Finally, we assess the isotropy assumption. Isotropy means that spatial dependence depends only on distance, not direction. In other words, two stations separated by the same distance should have similar dependence regardless of whether they are separated north-south, east-west, or diagonally.

```
# Directional variograms for assessing isotropy.
```

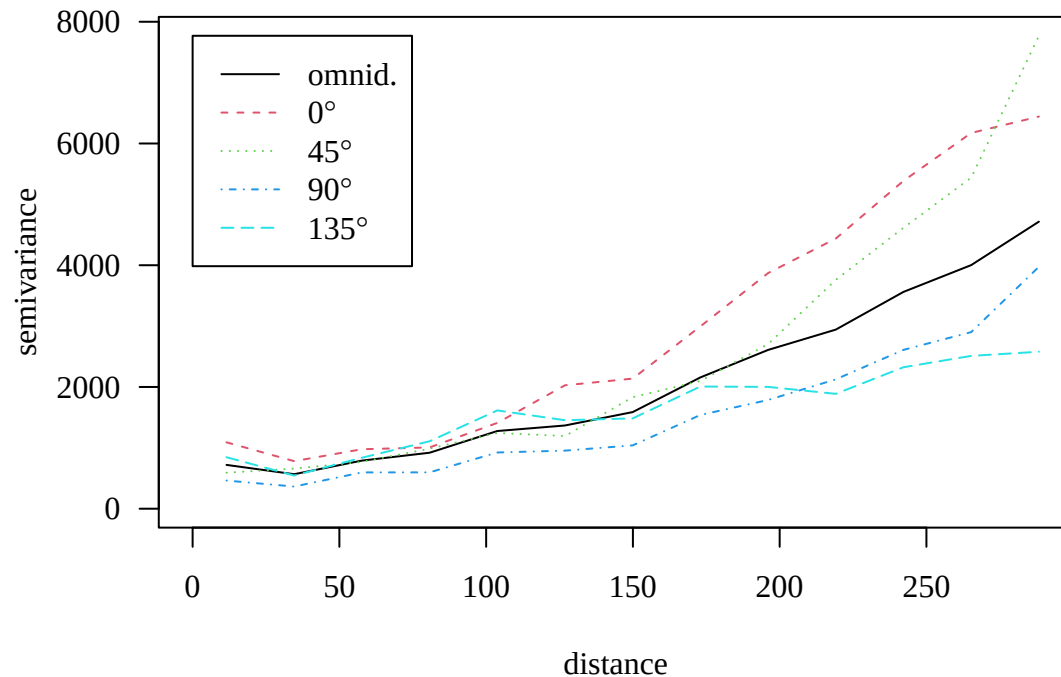
```
par(las = 1, family = "serif")
```

```
parana.v4 <- variog4(parana, max.dist = 300)
```

```
## variog: computing variogram for direction = 0 degrees (0 radians)
##      tolerance angle = 22.5 degrees (0.393 radians)
## variog: computing variogram for direction = 45 degrees (0.785 radians)
##      tolerance angle = 22.5 degrees (0.393 radians)
## variog: computing variogram for direction = 90 degrees (1.571 radians)
##      tolerance angle = 22.5 degrees (0.393 radians)
## variog: computing variogram for direction = 135 degrees (2.356 radians)
##      tolerance angle = 22.5 degrees (0.393 radians)
## variog: computing omnidirectional variogram
```



```
plot(parana.v4, env = parana.v.env, omni = TRUE)
```



The directional variograms allow us to compare spatial dependence across different directions. If the curves are broadly similar, the isotropy assumption is reasonable. If the curves differ substantially by direction, this may suggest anisotropy, meaning that the strength or range of spatial dependence changes with direction.

Overall, the variogram analysis helps justify the use of a Gaussian process spatial model. The increasing empirical variogram suggests positive spatial dependence, while the trend-adjusted variogram helps separate large-scale spatial trend from residual local spatial correlation. The Monte Carlo envelope provides a diagnostic check against spatial independence, and the directional variograms help evaluate whether isotropy is a reasonable modeling assumption.

Parameter Estimation

After examining the empirical variogram, we now fit parametric semivariogram models. The purpose is to estimate covariance parameters that will later be used for kriging.

Here we use the trend-adjusted empirical variogram, `parana.variot`, because the exploratory analysis suggested a large-scale spatial trend. By removing the first-order trend before fitting the variogram, we focus on modeling the residual spatial dependence rather than mixing trend effects with covariance effects.

```
# Fit two candidate semivariogram models to the trend-adjusted empirical variogram.
# The default model in variofit() is exponential.
parana.vtfit.exp <- variofit(parana.variot)
```

```
## variofit: covariance model used is matern
## variofit: weights used: npairs
## variofit: minimisation function used: optim
```

```
## Warning in variofit(parana.variot): initial values not provided - running the
## default search
```

```
## variofit: searching for best initial value ... selected values:
##          sigmasq  phi      tausq   kappa
## initial.value "1311.47" "230.66" "327.87" "0.5"
## status        "est"    "est"    "est"   "fix"
## loss value: 33524269.3444707
```

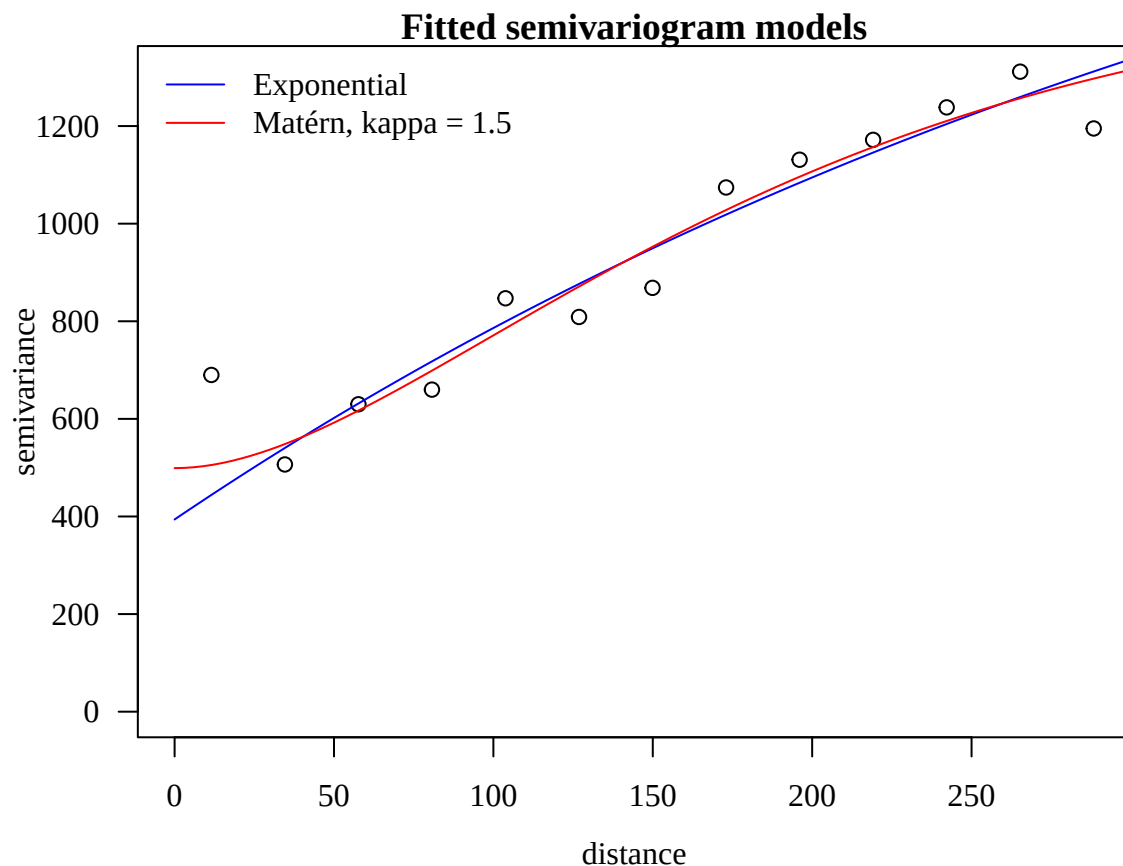
```
parana.vtfit.mat1.5 <- variofit(parana.variot, kappa = 1.5)
```

```
## variofit: covariance model used is matern
## variofit: weights used: npairs
## variofit: minimisation function used: optim
```

```
## Warning in variofit(parana.variot, kappa = 1.5): initial values not provided -
## running the default search
```

```
## variofit: searching for best initial value ... selected values:
##          sigmasq phi      tausq   kappa
## initial.value "983.6" "138.39" "655.73" "1.5"
## status        "est"    "est"    "est"   "fix"
## loss value: 43717205.8946468
```

```
# Plot the empirical variogram and overlay the fitted models.
par(las = 1, mar = c(3.5, 3.65, 1, 0.5), mgp = c(2.5, 1, 0), family = "serif")
plot(parana.variot, main = "Fitted semivariogram models")
lines(parana.vtfit.exp, col = "blue")
lines(parana.vtfit.mat1.5, col = "red")
legend("topleft", c("Exponential", "Matérn, kappa = 1.5"),
      col = c("blue", "red"), lty = 1, bty = "n")
```



The fitted curves summarize the estimated spatial dependence structure. The exponential model and the Matérn model provide two possible covariance specifications. The Matérn model is more flexible because it includes a smoothness parameter, here fixed at $\kappa = 1.5$.

These variogram-based estimates are useful for two reasons. First, they provide a direct visual assessment of how well the candidate covariance models match the empirical variogram. Second, they can provide reasonable starting values for likelihood-based estimation.

MLE

Variogram fitting is useful, but it only uses summarized pairwise information. Maximum likelihood estimation uses the full multivariate Gaussian model and therefore uses the full covariance structure among all observations.

We now fit two Gaussian process models using maximum likelihood. The first model uses a first-order spatial trend, while the second model uses a second-order spatial trend.

```
# Fit Gaussian process models by maximum likelihood.
# trend = "1st" includes a linear spatial trend in the coordinates.
# trend = "2nd" includes a quadratic spatial trend.
# ini = c(partial sill, range) gives initial covariance parameter values.
# nug gives the initial nugget value.
parana.ml1 <- likfit(parana, trend = "1st", ini = c(1000, 50), nug = 100)
```

```
## -----
## likfit: likelihood maximisation using the function optim.
```

```
## likfit: Use control() to pass additional
##      arguments for the maximisation function.
##      For further details see documentation for optim.
## likfit: It is highly advisable to run this function several
##      times with different initial values for the parameters.
## likfit: WARNING: This step can be time demanding!
## -----
## likfit: end of numerical maximisation.
```

```
parana.ml2 <- likfit(parana, trend = "2nd", ini = c(1000, 50), nug = 100)
```

```
## -----
## likfit: likelihood maximisation using the function optim.
## likfit: Use control() to pass additional
##      arguments for the maximisation function.
##      For further details see documentation for optim.
## likfit: It is highly advisable to run this function several
##      times with different initial values for the parameters.
## likfit: WARNING: This step can be time demanding!
## -----
## likfit: end of numerical maximisation.
```

```
parana.ml1
```

```
## likfit: estimated model parameters:
##      beta0      beta1      beta2      tausq      sigmasq      phi
## "416.4984" " -0.1375" " -0.3997" "385.5180" "785.6904" "184.3863"
## Practical Range with cor=0.05 for asymptotic range: 552.3719
##
## likfit: maximised log-likelihood = -663.9
```

```
parana.ml2
```

```
## likfit: estimated model parameters:
##      beta0      beta1      beta2      beta3      beta4      beta5      tausq
## "423.9282" " 0.0620" " -0.6360" " -0.0004" " 0.0000" " 0.0006" "381.2267"
##      sigmasq      phi
## "372.5993" " 77.5441"
## Practical Range with cor=0.05 for asymptotic range: 232.3013
##
## likfit: maximised log-likelihood = -660.2
```

The first-order trend model assumes that the mean rainfall surface changes linearly with the spatial coordinates. The second-order trend model allows additional curvature in the mean surface.

In practice, we compare these fitted models using the maximized log-likelihood, model complexity, and scientific interpretability. If the second-order model does not substantially improve the fit, the simpler first-order trend model may be preferred.

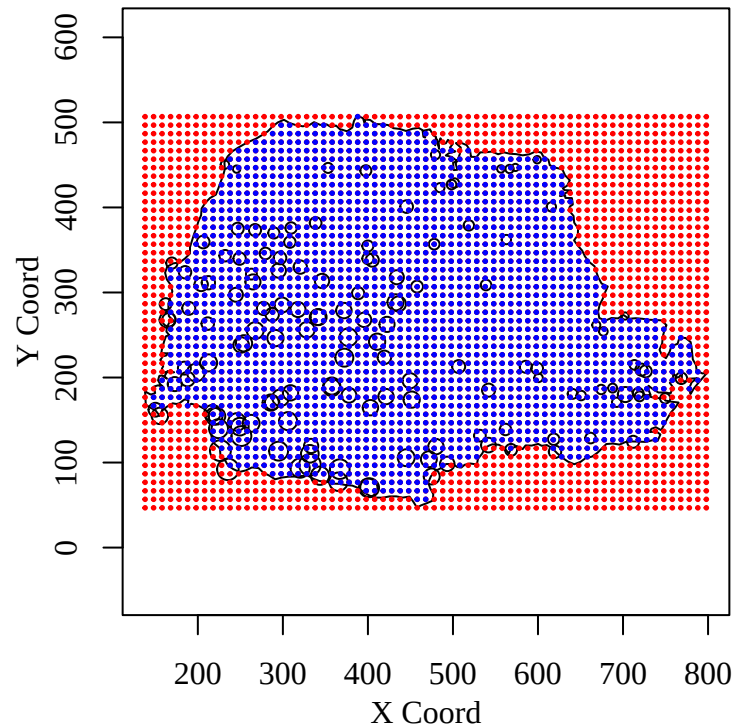
Spatial Prediction

After estimating the mean and covariance parameters, we can perform spatial interpolation. The goal is to predict rainfall at unobserved locations across Paraná State.

First, we create a regular grid of prediction locations. The red points represent the full rectangular grid, while the blue points represent grid points that fall inside the state boundary.

```
# Create a regular prediction grid and identify grid points inside the boundary.
parana.gr <- pred_grid(parana$borders, by = 10)
parana.gr0 <- locations.inside(parana.gr, parana$borders)

# Plot observed stations, full grid points, and grid points inside the boundary.
par(mgp = c(2, 1, 0), family = "serif")
points(parana)
points(parana.gr, pch = 19, col = "red", cex = 0.25)
points(parana.gr0, pch = 19, col = "blue", cex = 0.25)
```



The prediction grid defines where kriging will be performed. In principle, kriging can predict at any spatial location, but using a regular grid allows us to create a smooth prediction map.

Next, we specify the kriging model and run spatial prediction. We use the first-order trend maximum likelihood model, `parana.ml1`, because it is consistent with the exploratory trend analysis.

```
# Set up kriging using the fitted first-order trend model.
# trend.d specifies the trend in the observed data.
# trend.l specifies the trend at prediction locations.
KC <- krige.control(obj.m = parana.ml1, trend.d = "1st", trend.l = "1st")

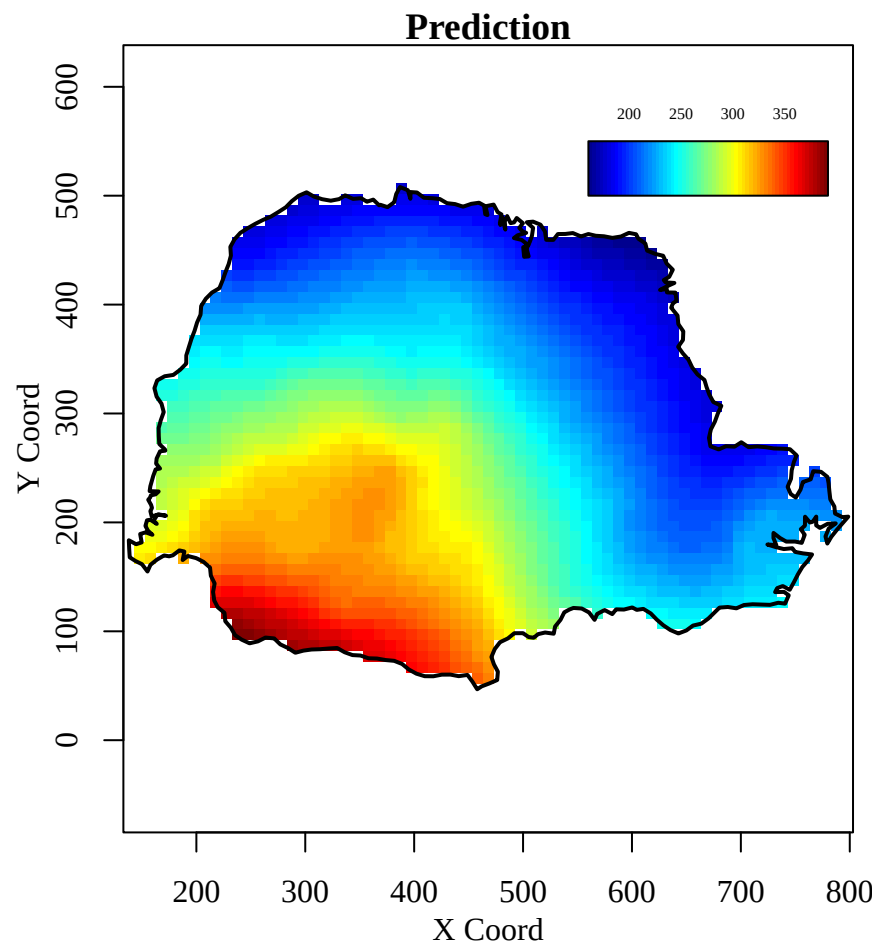
# Request conditional simulations in addition to kriging predictions.
OC <- output.control(simulations = TRUE, n.pred = 1000)

# Perform kriging over the prediction grid.
parana.kc <- krige.conv(parana, loc = parana.gr, krige = KC, output = OC)
```

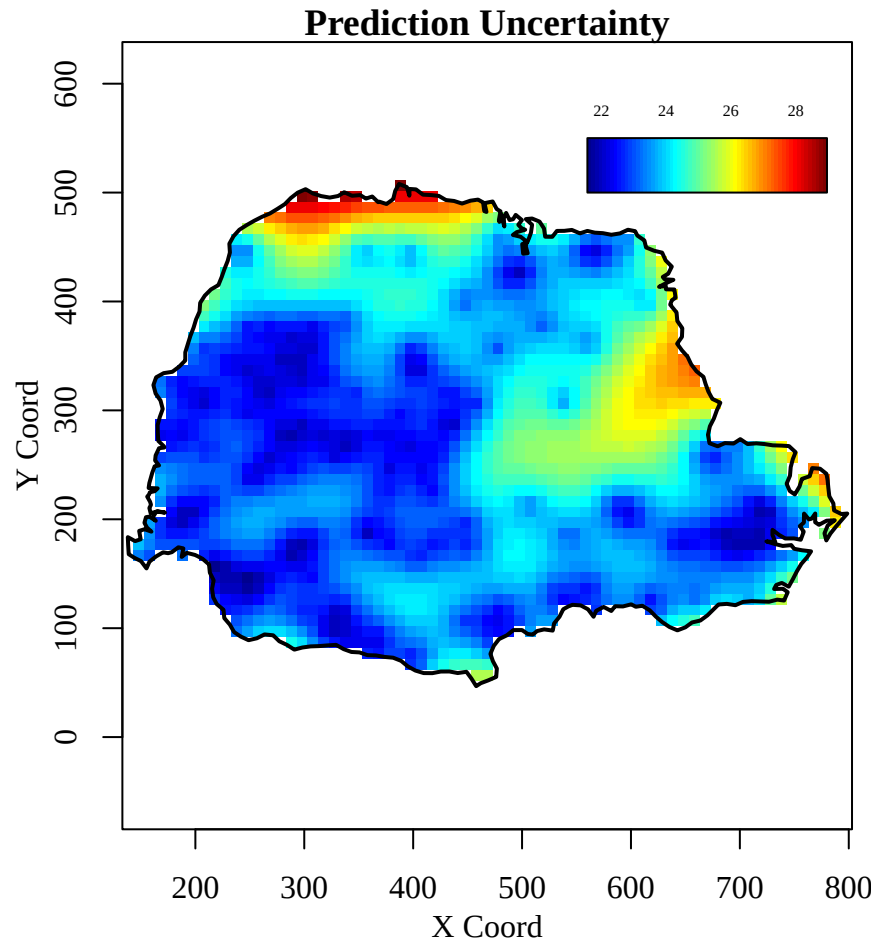
```
## krige.conv: results will be returned only for prediction locations inside the borders
## krige.conv: model with mean given by a 1st order polynomial on the coordinates
## krige.conv: sampling from the predictive distribution (conditional simulations)
## krige.conv: Kriging performed using global neighbourhood
```

The object `parana.kc` contains the kriging prediction, the kriging variance, and conditional simulations. The prediction represents the estimated rainfall surface, while the kriging variance measures prediction uncertainty.

```
# Plot the kriging prediction map.
par(family = "serif", mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0))
image(parana.kc, col = tim.colors(), x.leg = c(560, 780),
      y.leg = c(500, 550), cex = 0.5, main = "Prediction")
```



```
# Plot the prediction standard error map.
image(parana.kc, val = sqrt(parana.kc$krige.var), col = tim.colors(),
      x.leg = c(560, 780), y.leg = c(500, 550), cex = 0.5,
      main = "Prediction Uncertainty")
```



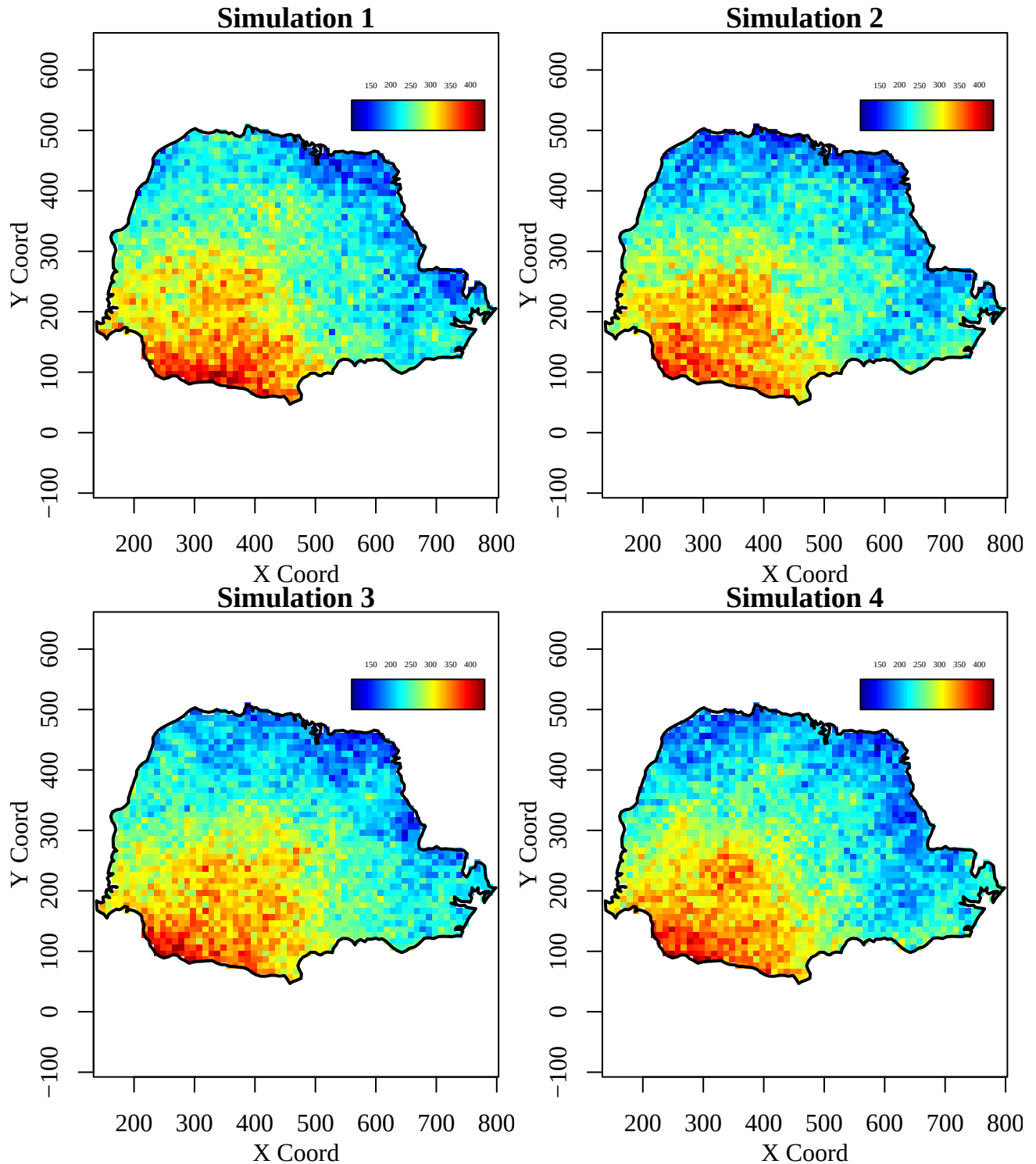
The prediction map shows the estimated spatial distribution of rainfall across Paraná State. The uncertainty map shows where predictions are more or less reliable. In general, uncertainty tends to be smaller near observed weather stations and larger in regions with fewer nearby observations or near the boundary of the spatial domain.

Conditional Simulations

Kriging gives the conditional mean surface, but it does not show the full range of possible spatial fields consistent with the observed data and fitted model. Conditional simulation addresses this by generating possible rainfall surfaces from the conditional distribution.

Each simulated surface honors the observed data and follows the estimated spatial covariance structure.

```
# Display four conditional simulations from the fitted Gaussian process model.
# These show possible rainfall surfaces conditional on the observed data.
par(family = "serif", mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0))
for (i in 1:4) {
  image(parana.kc, val = parana.kc$simulation[, i], col = tim.colors(),
        x.legend = c(560, 780), y.legend = c(500, 550), cex = 0.35,
        main = paste("Simulation", i),
        zlim = range(parana.kc$simulation[, 1:4]))
}
```



The conditional simulations show several plausible rainfall surfaces. They share the broad spatial pattern implied by the data and the fitted covariance model, but they also show local variation. This is useful because it reminds us that the kriging prediction is only an average surface, not the only possible surface.

Overall, this section completes the spatial interpolation workflow:

Explore data → Estimate variogram → Fit covariance model → Perform kriging → Map prediction and uncertainty.